

**A Summer Internship
Project Report
on
“Book A Doctor”
(MERN Stack Medical Appointment
Scheduler)**

Submitted in partial fulfilment of the
Requirements for the award of the Degree of
**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING**

By
Sriram Gandra

24EG105B13

Under the Guidance of

Mrs. Rama Mrudula



**Department of Computer Science
& Engineering ANURAG UNIVERSITY
GHATKESAR (M), MEDCHAL DISTRICT,
Hyderabad-500 088
Academic Year: 2025-2026**

ANURAG UNIVERSITY

Hyderabad-500 088

Department of Computer Science & Engineering



CERTIFICATE

I, Sriram Gandra, bearing hall ticket number, 24EG105B13, hereby declare that the project report entitled “**“Book A Doctor (MERN Stack Medical Appointment Scheduler)”**” Department of Computer Science & Engineering, Anurag University, Hyderabad, is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**.

This is a record of bonafide work carried out by me and the results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Signature of The Guide
Mrs. Rama Mrudula

Sriram Gandra
24EG105B13

ACKNOWLEDGMENT

I would like to express my sincere thanks and deep sense of gratitude to project supervisor Mrs. Rama Mrudula, for her constant encouragement and inspiring guidance without which this project could not have been completed. Her critical reviews and constructive comments improved our grasp of the subject and steered to the fruitful completion of the work. Her patience, guidance and encouragement made this project possible.

I would like to express my sincere gratitude to Dr. Archana Mantri, Vice Chancellor, Anurag University and Dr. P. Bhaskara Reddy, Registrar, Anurag University for their encouragement and support.

I would like to express my special thanks to Dr. V. Vijaya Kumar, Dean School of Engineering, Anurag University, for his encouragement and timely support in our B.Tech program.

I would like to acknowledge my sincere gratitude for the support extended by Dr. G. Vishnu Murthy, Dean, Department of Computer Science Engineering, Anurag University. We also express my deep sense of gratitude to Dr. P. Ravinder Rao, academic coordinator, whose research expertise and commitment to the highest standards continuously motivated us during the crucial stage of our project work.

Sriram Gandra

24EG105B13

DECLARATION

I hereby declare that the project entitled “**Book A Doctor (MERN Stack Medical Appointment Scheduler)**” submitted to the Department of Computer Science and Engineering is an original work carried out by us under the guidance of Mrs. Rama Mrudula. The work presented in this report has not been submitted elsewhere for the award of any degree.

Sriram Gandra

24EG105B13

ABSTRACT

This Summer Internship Project Report presents 'Book A Doctor', a full-stack web application developed using the MERN stack (MongoDB, Express, React, Node.js) to automate and streamline the medical appointment scheduling workflow. Traditional healthcare clinics suffer from administrative inefficiencies, double bookings, lack of direct file access, and long patient waiting times. 'Book A Doctor' addresses these challenges by introducing a role-based environment with three dedicated user dashboards: Patient, Doctor, and Administrator. Doctors post their consultation calendar slots; patients register, browse specialists, filter by clinics/specialization, book specific slots, and upload digital prescriptions and medical lab files. A security queue ensures that newly registered doctor accounts remain hidden from public search listings until reviewed and approved by an Administrator. Scalable architecture, custom JWT token authentication, and static binary file streams via Express/Multer ensure that the platform delivers a secure, responsive, and robust patient-provider coordination hub.

TABLE OF CONTENTS

TOPICS	PAGE NO.
Acknowledgement	i
Declaration	ii
Abstract	iii
1. Introduction	1
2. Problem Statement	2
3. Objectives	3
4. Scope of the Project	4
5. Existing System	5
6. Proposed System	6
7. Software & Hardware Requirements	7
8. System Design	8
9. Flowchart & Algorithm	10
10. Modules Description	11
11. Implementation	12
12. Results / Output Screens	13
13. Advantages & Limitations	17
14. Future Enhancements	18
15. Conclusion	19
16. References	20
17. Source Code	21

1. Introduction

In recent years, the integration of digital technology into the healthcare sector has transformed patient-provider interactions. Healthcare facilities are moving away from paper-based registries and manual scheduling systems towards secure, online portals. Real-time patient appointment scheduling platforms act as crucial bridges, ensuring that patient coordination is seamless and administrative staff workloads are reduced.

This project, titled 'Book A Doctor', represents a secure, scalable MERN-stack medical portal that digitizes the booking workflow. The frontend interface, developed using React and Vite, provides a responsive single-page application experience. The backend services, powered by Node.js and Express.js, serve clean REST API endpoints for user registrations, profile searches, availability management, booking state modifications, and document storage. The system incorporates robust authorization controls to segregate administrative access, specialist management, and patient record management.

2. Problem Statement

Medical offices and clinics frequently encounter bottlenecks due to outdated patient flow coordination methods. The key issues addressed by this system include:

- *Scheduling Conflicts: Double-bookings and incorrect calendar listings arise when staff handle booking sheets manually.*
- *Information Silos: Medical records, prescriptions, and lab test results are typically kept in physical files, which are prone to misplacement and slow down access during emergency visits.*
- *Verification Gaps: Medical listings on open directories often include unverified or credentials-lacking providers, posing risks to patient safety.*
- *High Administrative Overhead: Administrative staff spend significant time resolving scheduling conflicts, coordinating timings over calls, and manually tracking patient attendance.*

3. Objectives

The primary objectives of the 'Book A Doctor' platform are:

1. **Role-Based Workflows:** To implement three distinct, secure portals tailored to Patient, Doctor, and Administrator roles.
2. **Admin Verification Gate:** To create an approval workflow where registered doctors are not searchable by patients until the administrator validates their credentials and activates their profiles.
3. **Real-Time Calendar Management:** To enable doctors to dynamically create, edit, and post consultation slots, and allow patients to immediately reserve available slots.
4. **Digital Record Vault:** To allow patients to upload, store, and access their laboratory reports and prescriptions securely.
5. **Notification Alerts:** To implement a header-integrated notification feed alerting users of status changes (e.g., when a doctor approves/rejects an appointment).

4. Scope of the Project

The scope of this project covers the end-to-end design, implementation, and deployment of a clinical scheduling platform. It encompasses the following aspects:

- User Authentication & Registration: Secure onboarding with password hashing via bcryptjs and role assignment.
- Interactive Booking Engine: Dynamically filtered calendar slots allowing one patient per slot.
- Static Medical Document Storage: Binary file uploads parsed by Multer middleware, linked to MongoDB database references.
- Admin Dashboard Panel: Statistics tracking (user count, active bookings) and an approval queue to verify pending doctor listings.
- Notification System: Header feed alerting users immediately of status updates.

5. Existing System

The existing clinical scheduling system relies heavily on manual intervention. Patient lists are written in physical appointment books, and clinic staff must scan pages manually to identify open slots. Bookings are handled via phone calls, where staff cross-reference doctor availability in real-time. Medical documents, such as lab reports and prescriptions, are kept in folder cabinets at the clinic or home.

Drawbacks of the Existing System:

- Lack of real-time visibility: Patients must contact the front desk directly to check openings.
- Vulnerability to scheduling conflicts: Manual entry errors lead to double-bookings.
- Inefficient slot utilization: Cancelled slots are rarely re-allocated in time, causing clinic revenue loss.
- High risk of data loss: Paper files are easily lost, damaged, or accessed by unauthorized individuals.

6. Proposed System

The proposed digital booking portal resolves these administrative gaps. Patients can browse, filter by specialty, and reserve slots directly online. The application instantly checks for scheduling conflicts at the database level before locking in a booking, eliminating overlap. Registered doctor accounts remain hidden from patient searches until manually approved by an administrator, ensuring credential verification.

Furthermore, patients can upload digital medical records (prescriptions and lab reports) directly onto the server. These files are mapped to the patient's MongoDB record and served statically, ensuring they are instantly accessible by both the patient and their treating physician. The decoupled frontend-backend architecture guarantees responsive, high-speed interaction and minimizes server load.

7. Software & Hardware Requirements

Development and deployment of the platform require the following specifications:

Software Specifications

Software Parameter	Requirement Detail
Operating System	Windows 10/11, macOS, or Linux
Runtime Environment	Node.js (v18.x or higher)
Package Manager	npm (v9.x or higher)
Database	MongoDB Community Edition (v6.x) or MongoDB Atlas Cloud
Frontend Compiler/Bundler	Vite (v8.0.12) with React (v19.2.6)
Development Editor	Visual Studio Code
REST API Client	Postman / Thunder Client

Hardware Specifications

Hardware Parameter	Requirement Detail
Processor	Dual-Core Intel i3 / AMD Ryzen 3 or higher (i5/Ryzen 5 recommended)
RAM	4 GB minimum (8 GB recommended for concurrent database and local servers)
Disk Space	10 GB free space (Solid State Drive recommended for fast database reads)
Display Resolution	1280 x 720 minimum (for responsive UI testing)

8. System Design

System Architecture

The application follows the MERN stack architecture, which divides development into three core tiers: a React-based Single Page Application frontend, an Express.js API web server, and a MongoDB database layer.

1. **Presentation Tier (Client):** Built on React, routing is handled client-side using React Router. The Context API (AuthContext) manages the active session, storing JWT tokens and user information, and handles API authentication headers.
2. **Logic Tier (Server):** Built on Node.js and Express.js, this tier handles REST API route parsing, input validation, custom token middleware checking, file uploads processing via Multer, and global error handling.
3. **Data Tier (Database):** MongoDB serves as the persistent data store. Interactions are managed using Mongoose Object Document Mapper (ODM), enforcing schema validations, unique index constraints, and relational references via ObjectIds.

Database Schemas

The MongoDB database design is divided into four collections:

User Schema Table

Field Name	Type	Required	Unique / Default	Description
_id	ObjectId	Yes	Auto-generated	Unique identifier for the user
name	String	Yes	None	Full name of the user
email	String	Yes	Unique	Email used as primary login credential
password	String	Yes	None	Bcrypt-hashed password hash
role	String	Yes	Enum: Patient/Doctor/Admin	Defines access permissions and dashboard views
phoneNumber	String	No	None	Contact number of the user
notifications	Array	No	Default: []	Array of notification message strings

Doctor Schema Table

Field Name	Type	Required	Unique / Default	Description
_id	ObjectId	Yes	Auto-generated	Unique identifier for the doctor profile
user	ObjectId	Yes	Ref: User	Links doctor profile to base User credentials
specialization	String	Yes	None	Medical specialty (Cardiology, Dermatology, etc.)
experience	Number	Yes	None	Years of medical practice experience
fees	Number	Yes	None	Consultation fees for slots booking
about	String	No	None	Brief biography or summary text
location	String	No	None	Physical clinic address or hospital room
isApproved	Boolean	Yes	Default: false	Admin approval flag; profile hidden from search if false
availability	Array	No	Default: []	List of dates and slots arrays

Appointment Schema Table

Field Name	Type	Required	Unique / Default	Description
_id	ObjectId	Yes	Auto-generated	Unique identifier for the appointment
patient	ObjectId	Yes	Ref: User	Reference to the booking Patient User
doctor	ObjectId	Yes	Ref: Doctor	Reference to the booked Doctor profile
date	String	Yes	None	Selected booking date (YYYY-MM-DD)
timeSlot	String	Yes	None	Selected booking time slot (e.g. 10:00 AM)
status	String	Yes	Default: Pending	Enum: Pending, Approved, Rejected, Completed
reason	String	No	None	Brief description of symptoms or checkup purpose
notes	String	No	None	Optional comments left by the Doctor or Admin

Document Schema Table

Field Name	Type	Required	Unique / Default	Description
_id	ObjectId	Yes	Auto-generated	Unique identifier for the record entry
patient	ObjectId	Yes	Ref: User	Reference to the document owner Patient User
title	String	Yes	None	Custom title for the uploaded file
fileName	String	Yes	None	Unique filesystem name generated by Multer
filePath	String	Yes	None	Relative disk path for server file loading
uploadedAt	Date	Yes	Default: Date.now()	Timestamp of document upload

Data Flow Diagram (DFD) Description

Level 0 (Context Diagram): The 'Book A Doctor' portal sits at the center of the system. Three external entities interact with it:

- Patient: Sends registration inputs, search queries, booking slots, and file streams. Receives booking status reports and notification details.
- Doctor: Sends consultation schedules, slot responses, and notes. Receives booking request summaries and patient file links.
- Admin: Sends profile approval triggers and stats lookups. Receives doctor application listings and usage counts.

Level 1 DFD: Internal processes include: 1.0 User Authentication, 2.0 Doctor Profile Listing, 3.0 Slot Reservation, 4.0 File Upload Management, and 5.0 Administrative Auditing. These processes read from and write to the Users, Doctors, Appointments, and Documents collections respectively.

9. Flowchart & Algorithm

User Login & Security Routing Flowchart

Flow description: User clicks Login -> Input email & password -> Submit -> Express auth controller validates email exists in User collection -> Compare password with bcrypt hash -> If invalid, return 401. If valid, generate JWT token -> Return token and role to Client -> Client redirects to dashboard corresponding to role (Patient, Doctor, or Admin). If route is altered, RouteGuards intercept token verify and reject navigation.

Appointment Booking Logic

Algorithm for Patient Slot Booking:

Step 1: Patient selects an approved Doctor profile and views their availability array.

Step 2: Patient selects an available date and a time slot from the listed slots.

Step 3: Patient fills in the consultation reason and clicks Book Appointment.

Step 4: Frontend sends POST request to `/api/appointments` with JSON body and JWT authorization header.

Step 5: Backend middleware decodes token to fetch Patient ID, verifies roles, and searches for slot conflicts.

Step 6: If the slot is already booked, return 400 Bad Request error. If open, create a new Appointment document with status set to 'Pending'.

Step 7: Push notification messages to both the Patient's and Doctor's user records. Return 201 Created success response.

10. Modules Description

The application is divided into four main operational modules:

1. Authentication & Security Module

Handles signup, login, session validation, and password security. Incorporates bcryptjs to hash passwords on save, and generates JSON Web Tokens (JWT) for session persistence. Express security middlewares intercept and decode request headers to authorize access roles.

2. Patient Management Module

Enables patients to register, update profiles, search approved doctors, filter by specialization, and select consultation slots. Also features a Document Upload component that uses Multer middleware to process binary file uploads and map them to their profile, allowing doctors to view prescriptions or lab reports prior to consultation.

3. Doctor Scheduling Module

Provides doctors with dashboard statistics, incoming booking feeds, and calendar management tools. Doctors can add availability slots and change appointment status (Approve, Reject, Complete) with optional comments, triggering header notification alerts for patients.

4. Administrative Auditing Module

Provides the administrator with control over system registrations. New doctor profiles remain pending and hidden from public search until reviewed and approved. The panel displays statistics on total registered users, approved doctors, and active appointments.

11. Implementation

The project is structured as a decoupled monorepo, separating the React frontend and Express backend. In the backend, `server.js` starts the application on port 5000 and establishes a connection to MongoDB. Routing is split into specialized route maps: `authRoutes`, `doctorRoutes`, `appointmentRoutes`, `documentRoutes`, and `adminRoutes`. Custom middlewares handle route protection and role verification.

In the frontend, Vite compiles the React codebase. React Router handles client-side page routing. `AuthContext` maintains the login token and handles authorization checks. `Axios` intercepts outgoing requests to append token headers, and styled components ensure a clean, responsive user experience.

12. Results / Output Screens

The following figures show screenshots of the completed 'Book A Doctor' portal, showcasing the different user interfaces and functionalities:

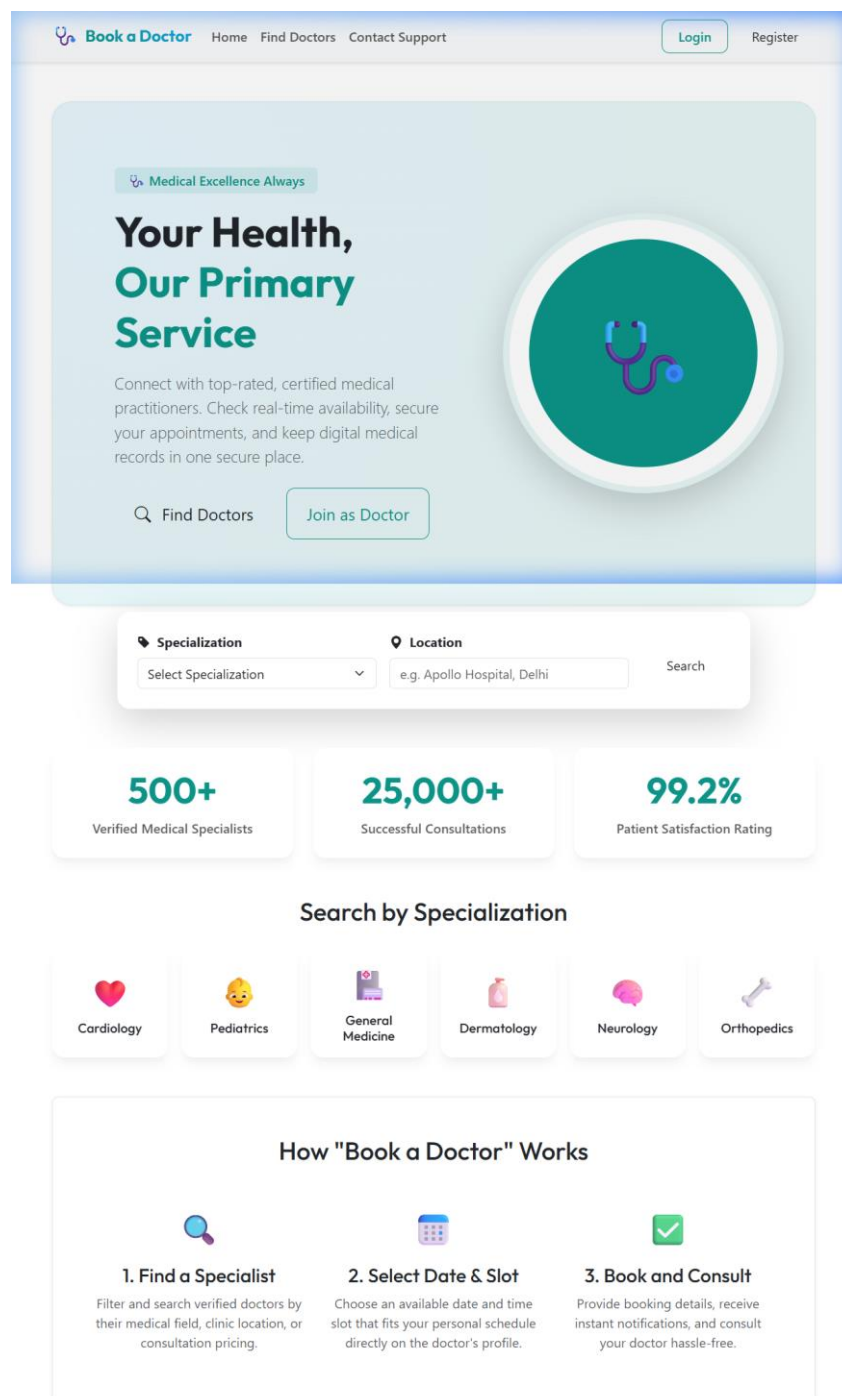
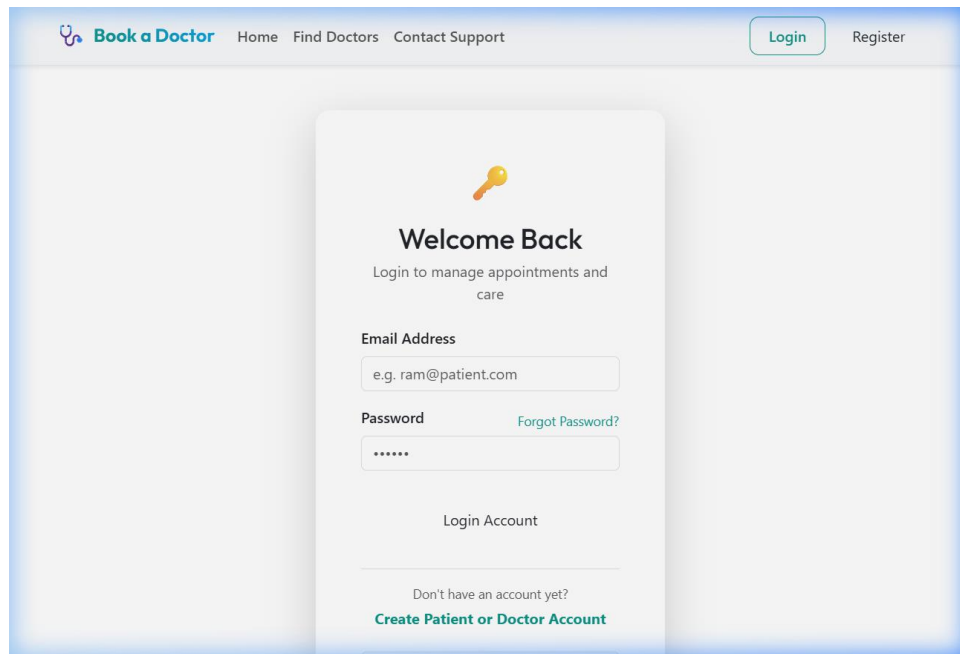


Figure 1: Home Page Specialist Search and Features Panel



The image shows a web interface for a medical booking system. At the top, there is a navigation bar with the logo "Book a Doctor" and links for "Home", "Find Doctors", and "Contact Support". On the right side of the navigation bar, there are "Login" and "Register" buttons. The main content area features a central white card with a yellow key icon at the top. The card is titled "Welcome Back" and includes the text "Login to manage appointments and care". Below this, there are input fields for "Email Address" (with a placeholder "e.g. ram@patient.com") and "Password" (with a placeholder "*****"). A link "Forgot Password?" is located next to the password field. Below the input fields, there is a "Login Account" button. At the bottom of the card, there is a link "Don't have an account yet?" and a button "Create Patient or Doctor Account".

Book a Doctor Home Find Doctors Contact Support Login Register

Welcome Back

Login to manage appointments and care

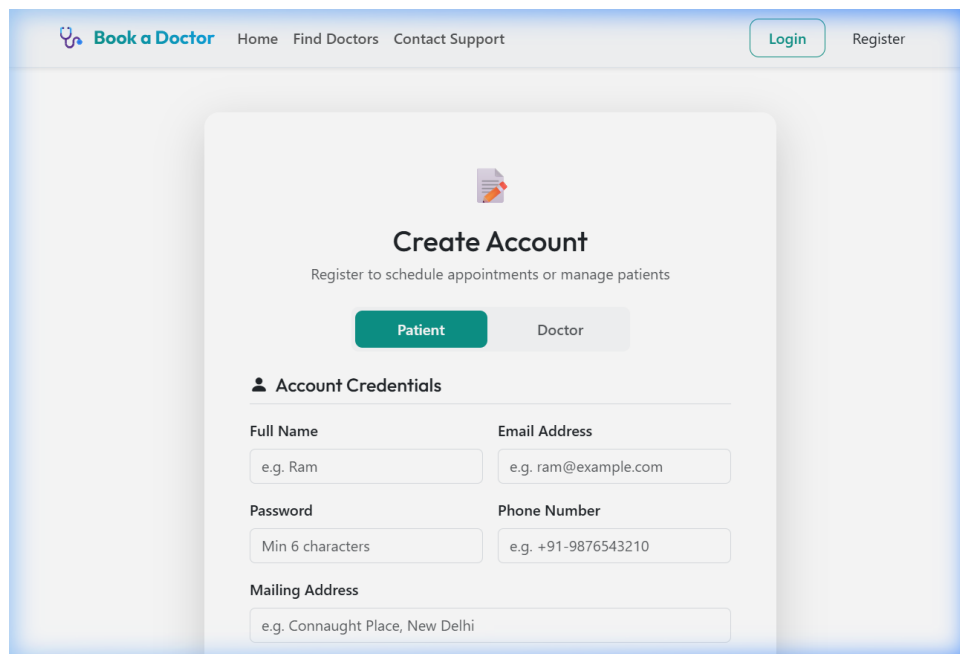
Email Address
e.g. ram@patient.com

Password [Forgot Password?](#)

Login Account

Don't have an account yet?
[Create Patient or Doctor Account](#)

Figure 2: User Login and Role Selection Interface



The image shows a web interface for a medical booking system. At the top, there is a navigation bar with the logo "Book a Doctor" and links for "Home", "Find Doctors", and "Contact Support". On the right side of the navigation bar, there are "Login" and "Register" buttons. The main content area features a central white card with a document icon at the top. The card is titled "Create Account" and includes the text "Register to schedule appointments or manage patients". Below this, there are two buttons: "Patient" (highlighted in green) and "Doctor". Below the buttons, there is a section titled "Account Credentials" with a list of input fields: "Full Name" (placeholder "e.g. Ram"), "Email Address" (placeholder "e.g. ram@example.com"), "Password" (placeholder "Min 6 characters"), "Phone Number" (placeholder "e.g. +91-9876543210"), and "Mailing Address" (placeholder "e.g. Connaught Place, New Delhi").

Book a Doctor Home Find Doctors Contact Support Login Register

Create Account

Register to schedule appointments or manage patients

Patient Doctor

Account Credentials

Full Name
e.g. Ram

Email Address
e.g. ram@example.com

Password
Min 6 characters

Phone Number
e.g. +91-9876543210

Mailing Address
e.g. Connaught Place, New Delhi

Figure 3: Doctor and Patient Registration Portal

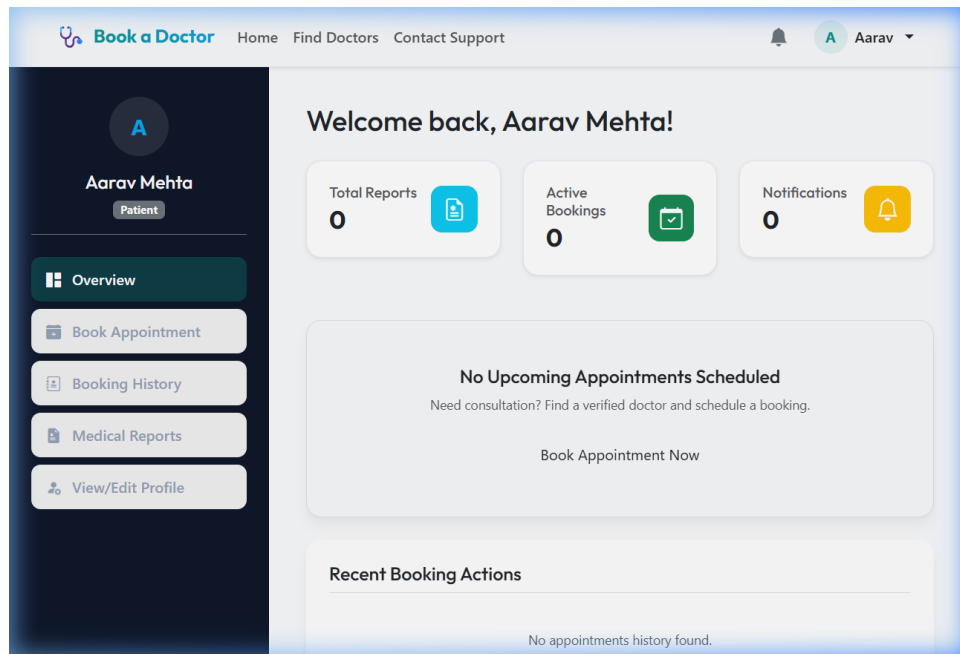


Figure 4: Patient Dashboard showing Bookings and Medical History

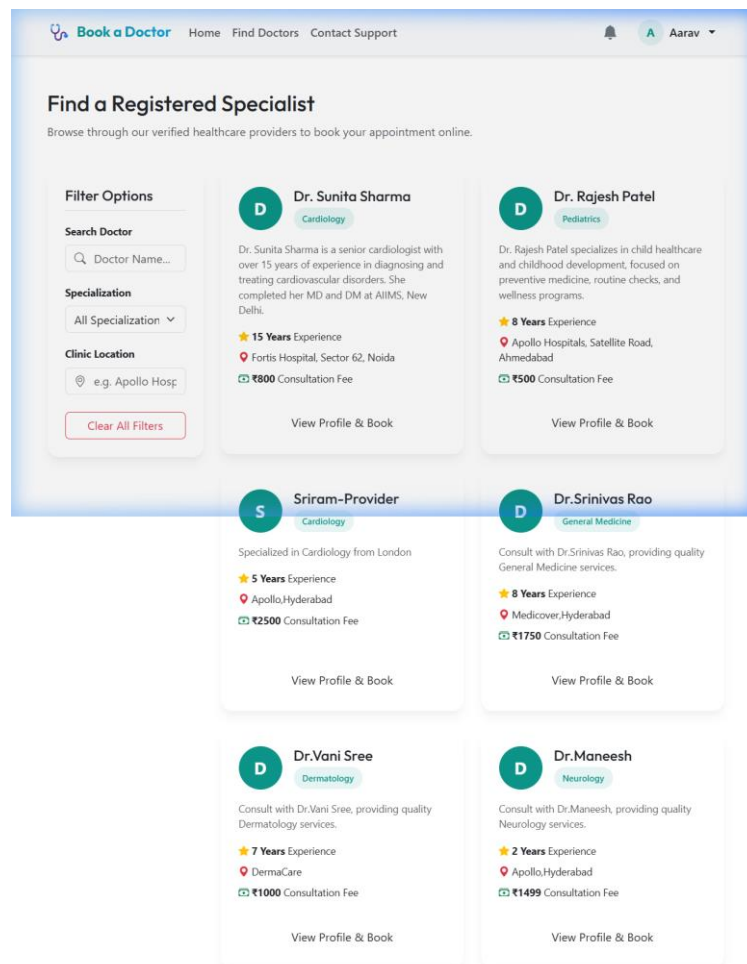


Figure 5: Specialist Directory filtered by Specialized Clinics

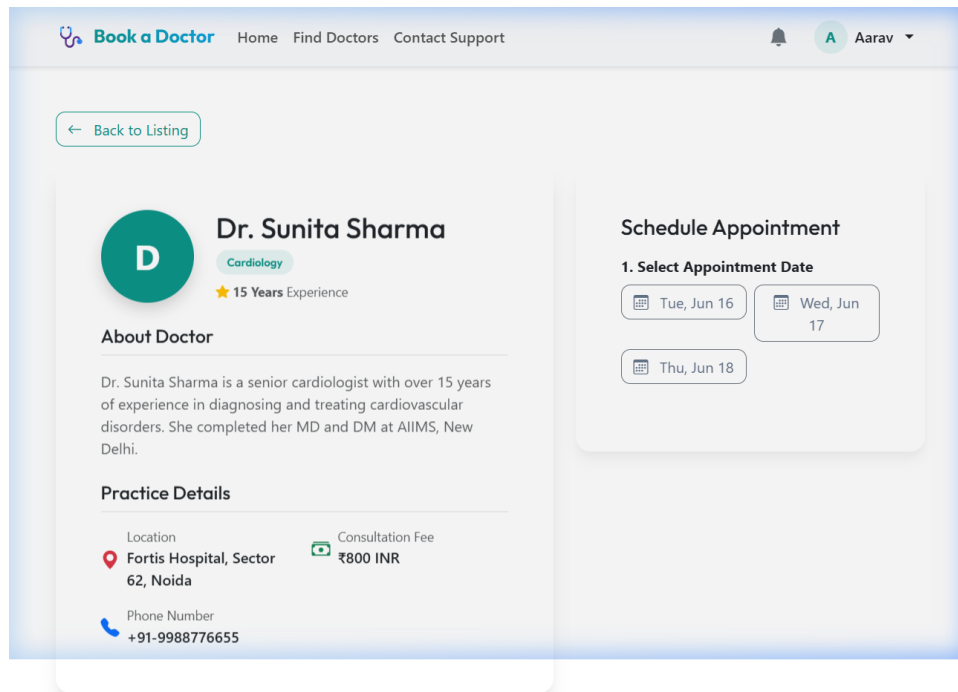


Figure 6: Doctor Profile Page with Interactive Slot Booking Grid

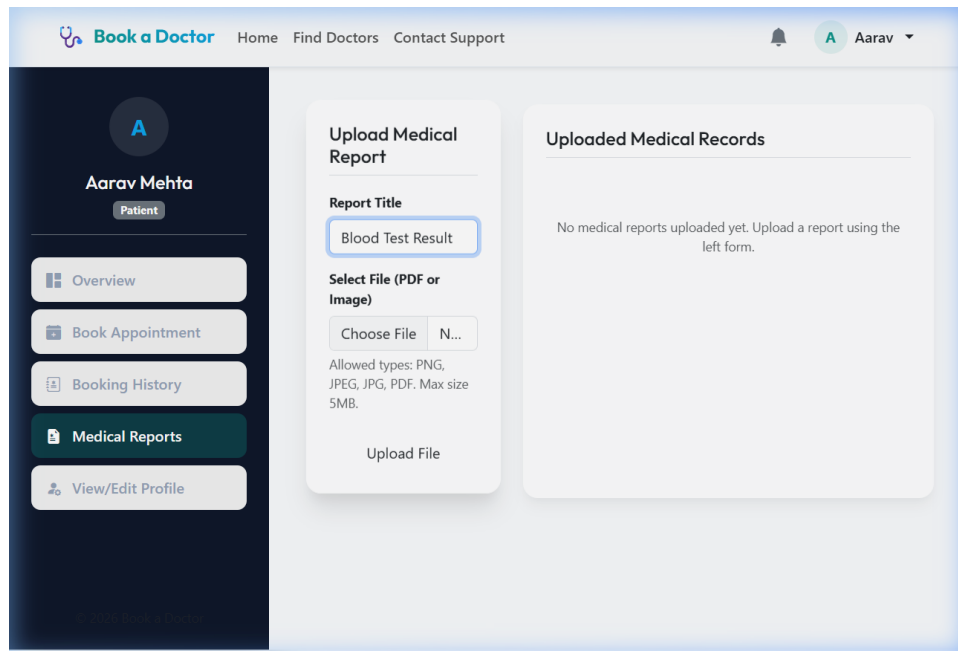


Figure 7: Medical Records Uploads and Prescriptions Registry Table

13. Advantages & Limitations

Advantages

- Real-time synchronization: Immediate slot updates prevent calendar booking overlaps.
- Decentralized workflow: Patients manage their records and bookings independently, lowering administrative overhead.
- Improved Security: Hashed credentials and JWT authorization prevent unauthorized data access.

Limitations

- Disk accumulation: Deleting a document from the MongoDB collection deletes the record, but the static binary remains in the server uploads directory.
- Concurrency limits: Simultaneous booking attempts by two patients at the identical millisecond can cause race conditions prior to model database writes.

14. Future Enhancements

To further scale the platform, future modules will include:

- WebRTC Integration: Real-time teleconsultation and secure video checks directly within the patient dashboard.
- Automated SMS Reminders: A Twilio gateway integration to alert patients of upcoming consultation slots.
- Payment Integration: Payment checkout checkout (Stripe or PayPal) directly during booking reservation.
- Automated static file cleanup: Cron job script to remove unreferenced medical upload files from disk.

15. Conclusion

The 'Book A Doctor' portal successfully digitizes the medical booking and patient management workflow. By implementing role-based dashboards, secure JWT session verification, static document mapping, and an admin verification gate, the application eliminates manual scheduling errors and improves record security. The project demonstrates how MERN stack technologies can be utilized to construct modular, secure, and highly performant full-stack systems.

16. References

1. React Documentation: Official guides on hooks, routing, and Context API. (<https://react.dev>)
2. Express Web Framework: Endpoint routing and middleware structure guides. (<https://expressjs.com>)
3. Mongoose ODM: Schemas, validations, and referenced relational structures. (<https://mongoosejs.com>)
4. JSON Web Tokens: Authentication specifications and JWT token layouts. (<https://jwt.io>)
5. Node.js Runtime: Server application execution guidelines. (<https://nodejs.org>)

17. Source Code

The following sections contain code listings for the core files of the 'Book A Doctor' application:

server/server.js: Server Entry Point (Express configuration and database connection startup)

Source Code: server/server.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const fs = require('fs');

const connectDB = require('./config/db');
const seedData = require('./config/seed');
const errorHandler = require('./middleware/error');

// Import routes
const authRoutes = require('./routes/authRoutes');
const doctorRoutes = require('./routes/doctorRoutes');
const appointmentRoutes = require('./routes/appointmentRoutes');
const documentRoutes = require('./routes/documentRoutes');
const adminRoutes = require('./routes/adminRoutes');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Ensure uploads folder exists and serve statically
const uploadsDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadsDir)) {
  fs.mkdirSync(uploadsDir, { recursive: true });
}
app.use('/uploads', express.static(uploadsDir));

// Mount Routes
app.use('/api/auth', authRoutes);
app.use('/api/doctors', doctorRoutes);
app.use('/api/appointments', appointmentRoutes);
app.use('/api/documents', documentRoutes);
app.use('/api/admin', adminRoutes);

// Serve static assets in production
if (process.env.NODE_ENV === 'production') {
  // Set static folder
  app.use(express.static(path.join(__dirname, '../client/dist')));

  app.get('*', (req, res, next) => {
    // If request starts with /api, pass it to api routes
    if (req.path.startsWith('/api')) {
      return next();
    }
    res.sendFile(path.join(__dirname, '../client/dist/index.html'));
  });
} else {
  // Root route
```

```

app.get('/', (req, res) => {
  res.json({ message: 'Welcome to Book a Doctor API' });
});
}

// Error Handler Middleware
app.use(errorHandler);

const PORT = process.env.PORT || 5000;

// Connect to DB and Start Server
connectDB().then(() => {
  // Seed initial data
  seedData();

  app.listen(PORT, () => {
    console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
  });
});

```

server/controllers/authController.js: Authentication Controller (handles user register, login, and profile fetching)

Source Code: server/controllers/authController.js

```

const crypto = require('crypto');
const User = require('../models/user');
const Doctor = require('../models/doctor');
const generateToken = require('../utils/generateToken');

// @desc    Register a new user
// @route    POST /api/auth/register
// @access   Public
exports.register = async (req, res, next) => {
  try {
    const {
      name,
      email,
      password,
      role,
      phoneNumber,
      address,
      // Doctor details
      specialization,
      experience,
      fees,
      about,
      location
    } = req.body;

    // Check if user already exists
    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ success: false, message: 'User already exists with this email' });
    }

    // Create User
    const user = await User.create({
      name,
      email,

```

```

        password,
        role: role || 'Patient',
        phoneNumber: phoneNumber || '',
        address: address || ''
    });

    let doctorProfile = null;

    // If role is Doctor, create Doctor profile
    if (user.role === 'Doctor') {
        if (!specialization || !experience || !fees || !location) {
            // Clean up created user if validation fails
            await User.findByIdAndDelete(user._id);
            return res.status(400).json({
                success: false,
                message: 'Please provide specialization, experience, fees, and location for
doctor registration'
            });
        }

        doctorProfile = await Doctor.create({
            user: user._id,
            specialization,
            experience,
            fees,
            about: about || '',
            location,
            isApproved: false // Requires admin approval
        });
    }

    const token = generateToken(user._id);

    res.status(201).json({
        success: true,
        token,
        user: {
            _id: user._id,
            name: user.name,
            email: user.email,
            role: user.role,
            phoneNumber: user.phoneNumber,
            address: user.address,
            profilePic: user.profilePic,
            doctorProfile
        }
    });
} catch (error) {
    next(error);
}
};

// @desc    Login user
// @route    POST /api/auth/login
// @access   Public
exports.login = async (req, res, next) => {
    try {
        const { email, password } = req.body;

        // Validate email & password
        if (!email || !password) {
            return res.status(400).json({ success: false, message: 'Please provide an email
and password' });
        }
    }

```



```

// Check for user
const user = await User.findOne({ email }).select('+password');
if (!user) {
  return res.status(401).json({ success: false, message: 'Invalid credentials' });
}

// Check if password matches
const isMatch = await user.matchPassword(password);
if (!isMatch) {
  return res.status(401).json({ success: false, message: 'Invalid credentials' });
}

let doctorProfile = null;
if (user.role === 'Doctor') {
  doctorProfile = await Doctor.findOne({ user: user._id });
}

const token = generateToken(user._id);

res.status(200).json({
  success: true,
  token,
  user: {
    _id: user._id,
    name: user.name,
    email: user.email,
    role: user.role,
    phoneNumber: user.phoneNumber,
    address: user.address,
    profilePic: user.profilePic,
    doctorProfile
  }
});
} catch (error) {
  next(error);
}
};

// @desc    Get current logged in user profile
// @route    GET /api/auth/me
// @access   Private
exports.getMe = async (req, res, next) => {
  try {
    const user = await User.findById(req.user.id);
    let doctorProfile = null;

    if (user.role === 'Doctor') {
      doctorProfile = await Doctor.findOne({ user: user._id });
    }

    res.status(200).json({
      success: true,
      user: {
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        phoneNumber: user.phoneNumber,
        address: user.address,
        profilePic: user.profilePic,
        notifications: user.notifications,
        doctorProfile
      }
    });
  } catch (error) {

```

```

    next(error);
  }
};

// @desc    Update user profile
// @route    PUT /api/auth/profile
// @access   Private
exports.updateProfile = async (req, res, next) => {
  try {
    const fieldsToUpdate = {
      name: req.body.name || req.user.name,
      phoneNumber: req.body.phoneNumber || req.user.phoneNumber,
      address: req.body.address || req.user.address,
      profilePic: req.body.profilePic || req.user.profilePic
    };

    const user = await User.findByIdAndUpdate(req.user.id, fieldsToUpdate, {
      new: true,
      runValidators: true
    });

    let doctorProfile = null;
    if (user.role === 'Doctor') {
      const docFields = {
        specialization: req.body.specialization,
        experience: req.body.experience,
        fees: req.body.fees,
        about: req.body.about,
        location: req.body.location
      };

      // Filter undefined fields
      Object.keys(docFields).forEach(key => docFields[key] === undefined && delete docFields[key]);

      doctorProfile = await Doctor.findOneAndUpdate(
        { user: user._id },
        { $set: docFields },
        { new: true, runValidators: true }
      );
    }

    res.status(200).json({
      success: true,
      user: {
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        phoneNumber: user.phoneNumber,
        address: user.address,
        profilePic: user.profilePic,
        doctorProfile
      }
    });
  } catch (error) {
    next(error);
  }
};

// @desc    Forgot password
// @route    POST /api/auth/forgot-password
// @access   Public
exports.forgotPassword = async (req, res, next) => {
  try {

```

```

    const user = await User.findOne({ email: req.body.email });

    if (!user) {
      return res.status(404).json({ success: false, message: 'There is no user with that email' });
    }

    // Generate reset token
    const resetToken = crypto.randomBytes(20).toString('hex');

    // Hash token and set to resetPasswordToken field
    user.resetPasswordToken = crypto
      .createHash('sha256')
      .update(resetToken)
      .digest('hex');

    // Set expire (10 mins)
    user.resetPasswordExpire = Date.now() + 10 * 60 * 1000;

    await user.save({ validateBeforeSave: false });

    // For test/dev purposes, return the reset token directly so client can consume it
    res.status(200).json({
      success: true,
      message: 'Token generated successfully',
      resetToken // In a real app we send this via email, here we expose it for ease of use
    });
  } catch (error) {
    next(error);
  }
};

// @desc    Reset password
// @route    PUT /api/auth/reset-password/:resettoken
// @access   Public
exports.resetPassword = async (req, res, next) => {
  try {
    // Hash token
    const resetPasswordToken = crypto
      .createHash('sha256')
      .update(req.params.resettoken)
      .digest('hex');

    const user = await User.findOne({
      resetPasswordToken,
      resetPasswordExpire: { $gt: Date.now() }
    });

    if (!user) {
      return res.status(400).json({ success: false, message: 'Invalid or expired token' });
    }

    // Set new password
    user.password = req.body.password;
    user.resetPasswordToken = undefined;
    user.resetPasswordExpire = undefined;
    await user.save();

    const token = generateToken(user._id);

    res.status(200).json({
      success: true,
      token,

```

```
    message: 'Password reset successful'
  });
} catch (error) {
  next(error);
}
};
```

server/models/doctor.js: Doctor Mongoose Schema (defines doctor details, billing, location, approval flag, and availability slots)

Source Code: server/models/doctor.js

```
const mongoose = require('mongoose');

const availabilitySchema = new mongoose.Schema({
  date: {
    type: String, // format YYYY-MM-DD
    required: true
  },
  slots: {
    type: [String], // e.g. ["09:00 AM", "10:00 AM"]
    required: true
  }
});

const doctorSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  specialization: {
    type: String,
    required: [true, 'Please add a specialization']
  },
  experience: {
    type: Number,
    required: [true, 'Please add experience in years']
  },
  fees: {
    type: Number,
    required: [true, 'Please add consultation fees']
  },
  about: {
    type: String,
    default: ''
  },
  location: {
    type: String,
    required: [true, 'Please add clinic or hospital location']
  },
  isApproved: {
    type: Boolean,
    default: false // Requires admin approval
  },
  availability: [availabilitySchema]
}, {
  timestamps: true
});

module.exports = mongoose.model('Doctor', doctorSchema);
```

client/src/App.jsx: Client Route Configuration (manages paths, role guards, and main layout rendering)

Source Code: client/src/App.jsx

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import ProtectedRoute from '../components/ProtectedRoute';

// Layout & Shared Components
import Navbar from '../components/Navbar';

// Public Pages
import Home from '../pages/Home';
import DoctorsListing from '../pages/DoctorsListing';
import DoctorDetails from '../pages/DoctorDetails';
import Login from '../pages/Login';
import Register from '../pages/Register';
import Contact from '../pages/Contact';

// Private Dashboards
import PatientDashboard from '../pages/PatientDashboard';
import DoctorDashboard from '../pages/DoctorDashboard';
import AdminDashboard from '../pages/AdminDashboard';

// Global Styles
import './App.css';

function App() {
  return (
    <Router>
      <AuthProvider>
        <div className="app d-flex flex-column min-vh-100">
          <Navbar />
          <main className="flex-grow-1">
            <Routes>
              { /* Public Routes */ }
              <Route path="/" element={ <Home /> } />
              <Route path="/doctors" element={ <DoctorsListing /> } />
              <Route path="/doctors/:id" element={ <DoctorDetails /> } />
              <Route path="/login" element={ <Login /> } />
              <Route path="/register" element={ <Register /> } />
              <Route path="/contact" element={ <Contact /> } />

              { /* Private Protected Routes */ }
              <Route
                path="/patient"
                element={
                  <ProtectedRoute allowedRoles={['Patient']}>
                    <PatientDashboard />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/doctor"
                element={
                  <ProtectedRoute allowedRoles={['Doctor']}>
                    <DoctorDashboard />
                  </ProtectedRoute>
                }
              />
            </Routes>
          </main>
        </div>
      </AuthProvider>
    </Router>
  );
}
```

```

        />
        <Route
          path="/admin"
          element={
            <ProtectedRoute allowedRoles={['Admin']}>
              <AdminDashboard />
            </ProtectedRoute>
          }
        />

        { /* Catch-all Redirect */ }
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </main>
  </div>
</AuthProvider>
</Router>
);
}

export default App;

```

client/src/context/AuthContext.jsx: Client Authentication Context (stores login states, tokens, and handles global login/logout redirects)

Source Code: client/src/context/AuthContext.jsx

```

import React, { createContext, useState, useEffect } from 'react';
import api from '../services/api';

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token') || null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Fetch current user details
  const loadUser = async () => {
    try {
      const res = await api.get('/auth/me');
      if (res.data.success) {
        setUser(res.data.user);
      } else {
        logout();
      }
    } catch (err) {
      console.error('Error loading user:', err);
      logout();
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    if (token) {
      localStorage.setItem('token', token);
      loadUser();
    } else {
      localStorage.removeItem('token');
      setUser(null);
    }
  }, [token]);

```

```

        setLoading(false);
    }
}, [token]));

// Register user
const registerUser = async (userData) => {
    setLoading(true);
    setError(null);
    try {
        const res = await api.post('/auth/register', userData);
        if (res.data.success) {
            setToken(res.data.token);
            setUser(res.data.user);
            return { success: true };
        }
    } catch (err) {
        const errMsg = err.response?.data?.message || err.response?.data?.error ||
'Registration failed';
        setError(errMsg);
        return { success: false, error: errMsg };
    } finally {
        setLoading(false);
    }
};

// Login user
const loginUser = async (email, password) => {
    setLoading(true);
    setError(null);
    try {
        const res = await api.post('/auth/login', { email, password });
        if (res.data.success) {
            setToken(res.data.token);
            setUser(res.data.user);
            return { success: true };
        }
    } catch (err) {
        const errMsg = err.response?.data?.message || err.response?.data?.error ||
'Invalid email or password';
        setError(errMsg);
        return { success: false, error: errMsg };
    } finally {
        setLoading(false);
    }
};

// Logout user
const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem('token');
};

// Update user profile
const updateProfile = async (profileData) => {
    setError(null);
    try {
        const res = await api.put('/auth/profile', profileData);
        if (res.data.success) {
            setUser(res.data.user);
            return { success: true };
        }
    } catch (err) {
        const errMsg = err.response?.data?.message || err.response?.data?.error ||
'Update profile failed';

```

```

        setError(errMsg);
        return { success: false, error: errMsg };
    }
};

// Refresh user data (like notifications count or profiles status)
const refreshUser = async () => {
    try {
        const res = await api.get('/auth/me');
        if (res.data.success) {
            setUser(res.data.user);
        }
    } catch (err) {
        console.error('Error refreshing user details:', err);
    }
};

return (
    <AuthContext.Provider
        value={{
            user,
            token,
            loading,
            error,
            register: registerUser,
            login: loginUser,
            logout,
            updateProfile,
            refreshUser,
            setError
        }}
    >
        {children}
    </AuthContext.Provider>
);
};

```